root, and so on till we get to a leaf: and that's exactly the position where we need to insert the key.

If you remember what we have seen in chapter 2 for heaps (or if you had a chance to refresh your memory on appendix C), all operations in a BST take time proportional to the height of the tree (the longest path from root to a leaf). In particular for balanced BSTs, all operations take `O(ln(n))` time, where `n` is the number of keys added to the tree.

Of course, compared to the `O(1)` amortized running time of hash tables, even balanced BSTs don't seem a good choice to implement an associative array. The catch is that, while their performance on the core methods is *slightly* slower, BSTs allow a substantial improvement for methods like finding the predecessor and successor o key, and finding minimum and maximum: they all run in `O(ln(n))` asymptotic time for BSTs, while the same operations on a hash table all require `O(n)` time.

Moreover, BSTs can return all keys (or values) stored, sorted by key, in linear time, while for hash tables you need to sort the set of keys after retrieving it, so it takes `O(M + n*ln(n))` comparisons.

Now that we have described the basic data structures most commonly used to implement dictionaries, it feels like a good time to recap what we have described so far: table 4.3 gathers the running times of the main operations on the possible implementations of dictionaries we mentioned.